

SD TCL - The Query Processor

Choosing records, sorting them, and printing a report, without writing a program.

The query processor is how you ask a file a question from the command line. It selects records, sorts them, formats them into columns and totals them, and it does all of that from one line of typing.

Twelve verbs on this page are the same program. `list`, `sort`, `count` and the rest are one catalogued program reached through fourteen VOC records, each carrying a number that says which verb was typed. Two of the fourteen — `select` and `sselect` — build a list instead of printing one and are covered in SD TCL - Select Lists.

SD folds case, so a query may be typed in either case. Commands and keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0. The file being queried is `voc`, because it is the one file every account has and it is interesting enough to ask real questions of.

The shape of a query

```
verb {dict} file {selection} {sort} {output fields} {modifiers}
```

Everything after the file name is optional, and the parts may be given in almost any order. The shortest useful query is a verb and a file:

```
count voc
```

```
418 record(s) counted
```

That number is particular to the account it was run in. Every account has a `voc` and no two need hold the same records — a standard account's is smaller than a programmer's, and `SDSYS`'s is different again. The counts on this page came from an administrator account.

The verbs

<code>list</code>	print the records
<code>sort</code>	print them in order
<code>count</code>	print how many there are, and nothing else
<code>sum</code>	total a field across the records
<code>list.item</code> · <code>sort.item</code>	print whole records, field by field, rather than in columns
<code>list.label</code> · <code>sort.label</code>	print in label format
<code>search</code>	select records by looking for text inside them
<code>show</code>	build a select list interactively
<code>reformat</code> · <code>sreformat</code>	write the selected fields out to another file

`sort` is `list` with an order imposed; every other pair works the same way, the `sort` half being the ordered one.

Choosing records

`with` introduces a selection clause:

```
count voc with dispatch # ""
```

```
144 record(s) counted
```

That one counts the verbs in the account, because `dispatch` is a dictionary entry that is empty for anything that is not a verb.

THE OPERATORS, AND THEIR SYNONYMS

SD accepts several spellings of each operator and they are exactly equivalent — the same token underneath. Each row below was run, and the spellings in a row returned identical counts on the same data.

equal

`=` · `eq`

not equal	# · ne · not
less than	< · lt · before
less or equal	<= · le
greater than	> · gt · after
greater or equal	>= · ge
matches a pattern	like · matches · matching
does not match	unlike · not.matching
sounds like	said · spoken

`before` and `after` read well on dates and are the same operators as `<` and `>`. All three spellings of *not equal* returned **331** on the same query, and `<`, `lt` and `before` all returned **381**.

JOINING CLAUSES

`and` (or `&`) and `or` join clauses. **Each clause repeats** `with`:

```
count voc with dispatch # "" and with dispatch # "CA"
```

```
47 record(s) counted
```

PATTERNS

`like` takes the same pattern language as SD BASIC's `matches` — `0x` for any number of characters, `1a` for one letter, `3n` for three digits, quoted text for a literal. It is set out in SD Basic - String Functions.

```
count voc with f1 like "V0X"
```

```
140 record(s) counted
```

A pattern that is not valid selects nothing and does not complain. Writing that same query as `like "V]"` returns **0 record(s) counted** — not an error, not a warning. **Always prove a pattern against a case you know matches** before believing a small answer, and be especially careful of a query that returns zero when you expected few.

OTHER WAYS TO CHOOSE

<code>every</code>	apply the test to every value of a multi-valued field
<code>when</code>	select values within a record rather than whole records
<code>no</code> <i>field</i>	records where the field is null
<code>unique</code>	one record per distinct value
<code>no.case</code>	compare without regard to case

Sorting

`by` sorts ascending, `by.dsnd` descending. Both take a field name, and both may be repeated for a second and third key.

```
sort voc with dispatch = "OS" by @id dispatch processor
```

```
@ID..... Dsp Processor.....
!          OS
sh         OS
2 record(s) listed
```

`@id` is the record id, and it is a field name like any other.

What gets printed

Name the fields you want after the selection, and they become the columns in the order you name them. With no fields named, the file's default listing is used — for `voc` that is a description built by the dictionary.

The column heading, its width and its format all come from the **dictionary** entry for the field, not from the query. `Dsp` above is the display name of the dictionary entry called `dispatch`.

<code>id.sup</code>	leave the record id out
<code>col.hdg</code> "text" · <code>display.name</code>	override a column heading

<code>fmt</code> <code>"code"</code>	override the format
<code>conv</code> <code>"code"</code>	override the conversion
<code>as</code> <code>name</code>	use another dictionary entry's formatting
<code>col.spaces</code> <code>n</code>	change the gap between columns
<code>vert</code>	one field per line instead of columns
<code>dbl.spc</code>	a blank line between records
<code>col.sup</code> · <code>col.hdr.sup</code> · <code>hdr.sup</code>	suppress column headings, both, or the page heading
<code>count.sup</code>	suppress the trailing record count
<code>det.sup</code>	suppress the detail lines, leaving totals

Headings and footings

By default the page heading is **the query you typed**, followed by a page number — which is genuinely useful on a printed report, because it says how the report was produced. `heading` replaces it:

```
sort voc with processor = "$QPROC" by @id dispatch processor heading "QP verbs"
```

QP verbs		
@ID.....	Dsp	Processor.....
count	CA	\$QPROC
list	CA	\$QPROC
list.item	CA	\$QPROC
list.label	CA	\$QPROC
reformat	CA	\$QPROC
search	CA	\$QPROC
select	CA	\$QPROC
show	CA	\$QPROC
sort	CA	\$QPROC
sort.item	CA	\$QPROC
sort.label	CA	\$QPROC
sreformat	CA	\$QPROC
sselect	CA	\$QPROC

```
sum          CA    $QPROC
14 record(s) listed
```

That is the query processor listing itself — all fourteen VOC records that reach it, including `count`, which is a keyword record that doubles as a verb.

`footing` does the same at the bottom of the page. Both accept the usual options inside the text for page number, date and time.

Totals, averages and breaks

<code>total</code> <i>field</i>	a column total
<code>average</code> · <code>avg</code>	the mean
<code>max</code> · <code>min</code>	the largest and smallest value
<code>enum</code> · <code>enumerate</code>	count the values rather than total them
<code>percent</code> · <code>pct</code> · <code>%P</code>	express as a percentage
<code>calc</code> · <code>calculate</code>	evaluate an I-type against the totals
<code>break.on</code> <i>field</i>	start a new subtotal group when the field changes
<code>break.sup</code> <i>field</i>	break on a field without printing it
<code>grand.total</code> "text" · <code>caption</code>	label the final total line

`break.on` is what turns a flat listing into a report with subtotals; the field it breaks on almost always wants to be a sort key as well, or the groups come out scattered.

Taking only some of the records

<code>sample</code> <i>n</i> · <code>first</code> <i>n</i>	the first <i>n</i> records
<code>sampld</code> <i>n</i>	every <i>n</i> th record

A sampled listing says so in its trailing line — `Sample of 3 record(s) listed` rather than `3 record(s) listed` — so a partial report cannot be mistaken for a whole one.

`sample` **TAKES THE FIRST RECORDS FOUND AND THEN SORTS THEM. IT IS NOT THE FIRST *n* IN SORTED ORDER.** Adding `sample 3` to the sorted listing above returns `count`, `show`, `sort` — not `count`, `list`, `list.item`, which is what the first three of that report actually are. **The sample is taken during selection and the sort happens afterwards, on the sample.** Use it to see the shape of a report cheaply; do not use it to find the top three of anything.

Where the output goes

<code>lptr {n}</code>	to print unit <i>n</i> , 0 by default
<code>crt</code>	to the screen, overriding a default of print
<code>no.page</code> · <code>nopage</code>	do not pause at the end of each screen
<code>file</code>	to a file rather than a device

Printing, print units and `$hold` are in SD TCL - Printing and Spooling.

Querying a dictionary

Put `dict` between the verb and the file name and the query runs against the dictionary instead of the data. It is the quickest way to find out what field names a file will accept:

```
list dict voc sample 14
```

@ID.....	TYPE	LOC.....	CONV..	NAME.....	FORMAT	S/M
@ID	D	0		@ID	20L	S
NAME	D	0		NAME	16L	S
F1	D	1		F1	5L	S
DATA.NAME	D	2		DATA Pathname	18L	M
...						

Sample of 14 record(s) listed

D entries name a field by position, **I** entries compute a value, and **PH** entries are phrases — stored fragments of query that expand where they are named.

Who has these verbs

standard

`count` `list` `list.item` `list.label` `search` `show` `sort` `sort.label` `sum`

programmer

`reformat` `sreformat` `sort.item`

Reading and reporting are available to every account; the two verbs that **write** their result into another file are not.

See also

SD TCL - Select Lists · SD TCL - Files and Records · SD Basic - String Functions.